Task 3: Product Detail, Persistent Cart, and Save-for-Later

Phase 2, Task 3 deliverable. Author: Ahmed. Date: 2026-11-12. Branch: `feature/product-detail-cart-save-for-later`.

Try it: `npm install`, `npm run start`, then open http://127.0.0.1:8080/catalog.html, choose a product, add it to the cart and open **Cart** in the menu. `product.html?id=usb-c-hub` is out of stock; `product.html?id=desk-lamp` has only 3 left.

1. What Was Built

Requirement from the brief	Where	Evidence
Product detail: image gallery	`src/components/gallery.js` (thumbnails with `aria-pressed`, arrow keys)	e2e journey 1
Product detail: specs table	`createSpecs()` in `src/components/product-detail.js` (<table> with row headers)	e2e journey 1
Product detail: stock status, quantity stepper, add to cart	`stockStatus()`, `src/components/quantity-stepper.js`, `createPurchase()`	e2e journeys 1, 2
Cart as pure functions over a state object: add, update quantity, remove, clear	`src/state/cart.js`: `addToCart`, `updateQuantity`, `removeFromCart`, `clearCart` (+ `canAdd`)	unit tests with deep-frozen input
Totals, shipping rule	`src/utils/cart-math.js`: `totals`, `shippingFor`, `clampQuantity`	8 fixture cases (section 3)
One currency format with `Intl.NumberFormat`	`formatMoney()` in `src/utils/money.js`, used by every price on every page (the Phase 1 home cards were switched to it too)	e2e journey 8
Cart in localStorage, rehydrated on load	`createCart()` + `rehydrate()` in `src/state/cart.js`	unit tests; e2e journey 4
Save-for-later list (add, move to cart, remove), persisted separately	`src/state/saved.js`: `saveItem`, `moveToCart`, `moveToSaved`, `removeSaved`, `createSaved`	unit tests; e2e journey 5
Recently viewed in sessionStorage, rendered as a strip	`src/state/recent.js` (`addRecent`, `recordView`), `src/components/recent-strip.js`	unit tests; e2e journey 6
Destructuring and spread in every state update	all functions in `state/cart.js`, `state/saved.js`, `state/recent.js` return new objects built with `...`; parameters are destructured	unit tests pass deep-frozen states (a mutation would throw)
One API module with Axios and a request interceptor	`src/api/product-api.js` (vendored Axios 1.19.0 in `src/vendor/axios.js`)	unit tests (interceptor headers, error conversion); integration test against the mock
Why the rest uses fetch	section 6	

2. Cart State and Pure Functions

State objects: `{ lines: CartLine[] }` and `{ items: CartLine[] }`, where a line is `{ productId, name, unitPrice (cents), currency, quantity, stock, image }`.

Function	Returns	Rule
`addToCart(cart, product, qty)`	new cart	new line, or more of an existing line; refused (same state back) when `canAdd` says no
`canAdd(cart, product)`	`{ ok, reason }`	out of stock → "Out of stock: this item cannot be added…"; at the limit → "You already have the most…"
`updateQuantity(cart, id, qty)`	new cart	quantity clamped to 1..min(stock, 10)
`removeFromCart(cart, id)` / `clearCart(cart)`	new cart	
`moveToSaved({cart, saved}, id)` / `moveToCart({cart, saved}, id)`	both new states	an item is never in both lists or lost; `moveToCart` is refused with a reason when out of stock
`syncWithCatalog(cart, products)`	new cart (or the same one if nothing changed)	applies current API price and stock; a product the API no longer has becomes unavailable

Function	Returns	Rule
`totals(lines)`	`{ itemCount, subtotal, shipping, total, freeShippingRemaining }`	integer cents

The store (`createCart`) only does three things around the pure functions: save the new state, notify listeners, and listen for the `storage` event so another open tab updates too.

3. Totals: Rule and Fixture Check

Shipping is **free from a $100.00 subtotal, otherwise $7.99**; an empty cart has no shipping; no tax in this version (the same rule as the API's orders). All money is integer cents; formatting happens only in `formatMoney`.

`tests/fixtures/cart-cases.json` holds reference totals computed separately with Python's `Decimal`, in dollars. The unit tests and the e2e journey compare the app's totals with them within **$0.005**.

Fixture case	Subtotal	Shipping	Total
empty cart	$0.00	$0.00	$0.00
one item below free shipping (wool socks)	$24.00	$7.99	$31.99
quantity 3 of one item (card wallet)	$89.97	$7.99	$97.96
just below the $100 threshold	$98.99	$7.99	$106.98
just above the $100 threshold	$101.00	$0.00	$101.00
two lines, one with quantity 2	$499.97	$0.00	$499.97
maximum quantity on one line (10 headphones)	$1,999.90	$0.00	$1,999.90
four lines, one at its stock limit	$499.48	$0.00	$499.48

All 8 match in the unit tests. The two threshold cases are also checked in the browser (e2e journey 3).

4. What Survives What

Data	Storage	Key	Refresh	Tab close / return visit	New tab
Cart	localStorage	`brandstore:cart:v1`	kept	kept	shared
Saved for later	localStorage (separate key)	`brandstore:saved:v1`	kept	kept	shared
Recently viewed (max 6, newest first, no duplicates)	sessionStorage	`brandstore:recent:v1`	kept	cleared	not shared

Stored data is validated when it is read back (`rehydrate`, `rehydrateSaved`, `readRecent`): malformed entries are dropped and quantities re-clamped, so an edited or corrupt value cannot break the page. Blocked or full storage never throws (`src/utils/storage.js`). When the cart page opens, prices and stock are re-checked with the API; a changed price is applied and announced, e.g. "Ergonomic Smartwatch is now $149.99 (was $1.00)."

5. Out-of-Stock Behaviour

Where	What happens
Product page	Status "Out of stock"; quantity stepper hidden; "Add to cart" disabled and linked (`aria-describedby`) to "This item is out of stock, so it cannot be added to the cart. Save it for later and check back soon."
Pure functions	`canAdd` returns the reason; `addToCart` and `moveToCart` refuse and return the unchanged state
Saved list	"Move to cart" disabled with the reason shown under the item
Cart (item sold out since it was added)	Line stays visible with "Out of stock: not included in the total. Save it for later or remove it."; it is left out of the totals

Where	What happens
Low stock	"Only 3 left in stock"; the stepper stops at the stock level

6. Axios in One Module, fetch Everywhere Else

`src/api/product-api.js` is the only Axios module. It serves the product page and the cart's price/stock refresh (`GET /products/{id}`). Its **request interceptor** adds `Accept: application/json` and an `X-Client-Id` for this tab and stamps the start time; its **response interceptor** records the duration and turns every failure into the same `ApiError` (status, code, message, details, Retry-After) that the fetch modules throw, so components handle both identically.

The rest of the app uses `fetch` (`src/api/http.js`) because:

- it is built into every browser: 0 KB, against 48 KB for Axios;
- it works with `AbortController` directly, which the catalog needs to cancel superseded searches;
- the catalog needs nothing Axios adds (no upload progress, no automatic transforms beyond JSON).

Axios is loaded as a plain ES module copied from the npm package (`npm run vendor`; version pinned in `package.json`), because the site has no bundler.

7. QA Matrix

Journey (`npm run e2e`)	Chrome desktop 1280	Chrome phone 375	Firefox	Safari / WebKit
1. Gallery, specs table, stock status; stepper limited by stock (desk lamp: 3)	Pass	Pass	Not run	Not run
2. Out-of-stock product cannot be added and explains why; saved copy cannot move	Pass	Pass	Not run	Not run
3. Totals match the fixtures just below and above free shipping	Pass	Pass	Not run	Not run
4. Cart survives refresh and a return visit (new tab); + updates the total	Pass	Pass	Not run	Not run
5. Save for later and back (quantity kept); lists persist separately; clear cart	Pass	Pass	Not run	Not run
6. Recently viewed: no duplicates, capped at 6, not shared with a new tab	Pass	Pass	Not run	Not run
7. Stale stored price corrected from the API and announced	Pass	Pass	Not run	Not run
8. One price format ($1,234.56) on catalog, product, cart and home pages	Pass	Pass	Not run	Not run
Catalog and Phase 1 journeys (18)	17 of 17	17 of 17	Not run	Not run

Total: **50 of 50 journey runs pass** in Chromium. Changing the shipping fee to $8.00 makes journey 3 fail, so the journeys catch wrong totals.

Other check	Result
`npm run lint` / `npm run format:check`	0 problems / all formatted
Unit tests (100) and integration tests (25)	125 of 125 pass (Node test runner through a Vitest-compatible shim; see deviation 2)
`npm run contract` (spec now includes the gallery `images`)	50 requests, 0 problems
`npm run build`	OK: 53 modules reachable from 6 pages
`node qa/js-rules.js` (ES modules, no inline scripts, no globals, functions ≤ 40 lines)	0 failures (59 files, 189 functions; `src/vendor` excluded)
`node qa/preflight.js` (accessibility and SEO, 6 pages)	322 of 322
`node qa/headers.js` (Netlify headers + CSP; product page uses Axios, cart page, catalog)	0 failures
`node qa/cls.js`	layout shift 0 (the "Cart (0)" link text is in the HTML, so the count does not move the menu)
Phase 1 behaviour, layout, keyboard, device matrix, contrast	all pass

8. Deviations List

#	Deviation	Reason
1	Axios is vendored into `src/vendor/axios.js` (exact copy of the npm package's ES-module build) instead of imported by name	The site runs without a bundler; browsers cannot resolve `import 'axios'`. `npm run vendor` refreshes the copy
2	Unit and integration tests were run with Node's test runner through a small Vitest-compatible shim; MSW not used	Vitest and MSW could not be installed in the build environment; `npm test` runs the same files with real Vitest
3	No tax	Not in the brief or the API; the summary says "No tax is charged"
4	Gallery images 2 and 3 are generic placeholders ("Side view", "What's in the box")	The catalog only has one photo per product
5	Checkout button is present but disabled	Checkout is a later task
6	Firefox, Safari and real devices not tested	Not available in the build environment